

Quote PDF Otomatik Üretim — Mimari

URLA SHOES — Salesforce Sandbox — Takım Sunum Dokümanı

Kritik tasarım notu: Bu çözümde bilinçli olarak Apex trigger **kullanılmıyor**. Salesforce'un yerli sunucu tarafı PDF üretim aracı Visualforce'tur (`PageReference.getContentAsPDF()`) ve proje gereği kapsam dışıdır. Bu sebeple PDF üretimi **tarayıcıda**, jsPDF kütüphanesi ile yapılır; otomatikleştirme de record trigger yerine bir polling mekanizmasıyla sağlanır.

1. Bileşenler

Katman	Bileşen	Görev
Tarayıcı	LWC <code>quotePdfGenerator</code>	Polling ile kayıt değişimini algılar, jsPDF ile PDF üretir, Apex'e yükleme isteği gönderir.
Static Resource	<code>jsPDF</code> (UMD min, ~365 KB)	Tarayıcı tarafında çalışan PDF üretim kütüphanesi.
Apex Service	<code>QuotePdfService</code>	4 metot: change signature, taze veri çekimi, versiyon numarası, attachment yükleme.
SObject	<code>Quote</code> , <code>QuoteLineItem</code> , <code>Attachment</code>	Standart objeler; üretilen PDF'ler Notes & Attachments'a düşer.

2. Uçtan Uca Akış

1 Kullanıcı Quote sayfasını açar

- LWC sayfaya bağlanır → `connectedCallback()` çalışır
- `loadScript(jsPdfResource)` kütüphaneyi indirir/cache'den alır
- İlk `pollOnce()` bir *baseline* (taban) signature kaydeder
- `setInterval(pollOnce, 3000)` polling döngüsünü başlatır

force-app/main/default/lwc/quotePdfGenerator/quotePdfGenerator.js (connectedCallback)

2 Kullanıcı "Add Products" ile ürün ekler ve kaydeder

- Salesforce `QuoteLineItem` kayıtlarını insert eder
- Quote rollup alanları sunucu tarafında güncellenir: `Subtotal`, `Discount`, `Tax`, `TotalPrice`, `GrandTotal`
- `Quote.LastModifiedDate` ilerler
- Tarayıcı bu noktada henüz hiçbir şey bilmez

3 Polling değişimi algılar (yaklaşık 3 sn içinde)

```
async pollOnce() {
  const signature = await getQuoteChangeSignature({ quoteId: this.recordId });
  if (this.previousSignature && this.previousSignature !== signature) {
    this.scheduleRegenerate(); // ← TETİKLEME NOKTASI
  }
  this.previousSignature = signature;
}
```

Apex metodu 7 parçadan oluşan sıkıştırılmış bir signature (parmak izi) string'i döner:

```
@AuraEnabled
public static String getQuoteChangeSignature(Id quoteId) {
  Quote q = [SELECT LastModifiedDate, Subtotal, Discount, Tax,
                TotalPrice, GrandTotal FROM Quote WHERE Id=:quoteId];
  Integer lineCount = [SELECT COUNT() FROM QuoteLineItem WHERE QuoteId=:quoteId];
  return q.LastModifiedDate.getTime() + '|' + q.Subtotal + '|' + q.Discount
    + '|' + q.Tax + '|' + q.TotalPrice + '|' + q.GrandTotal
    + '|' + lineCount;
}
```

QuotePdfService.cls — getQuoteChangeSignature

4 Debounce (1.5 saniye bekleme)

```
scheduleRegenerate() {
  if (this.regenerateTimeoutId) clearTimeout(this.regenerateTimeoutId);
  this.regenerateTimeoutId = setTimeout(async () => {
    await this.handleGenerate();
  }, 1500);
}
```

Neden debounce? Add Products wizard'ı birbiri ardına birden fazla DML üretebilir. Debounce, son save'den 1.5 sn sonrasına kadar bekler. Böylece 5 ürün eklediğinde 5 PDF yerine **tek** PDF üretilir.

5 Asıl üretim — handleGenerate()

```
async handleGenerate() {
  this.isGenerating = true;
  // 1. En taze veriyi çek (Apex'te cacheable=false olduğu için cache'siz)
  const freshData = await getQuoteData({ quoteId: this.recordId });
  // 2. Bir sonraki versiyon numarasını al
  const version = await getNextVersionNumber({ quoteId: this.recordId });
  // 3. PDF'i tarayıcıda jsPDF ile üret
  const base64 = this.buildPdfBase64(freshData, version);
}
```

```
// 4. Apex'e gönder; Attachment olarak kaydedilir
const fileName = `${freshData.quote.QuoteNumber}_v${version}.pdf`;
await uploadPdf({ quoteId: this.recordId, base64Body: base64, fileName });
// 5. Kullanıcıya toast göster
this.dispatchEvent(new ShowToastEvent({ title: 'PDF generated', ... }));
}
```

6 PDF render (jsPDF, tarayıcı tarafında)

`buildPdfBase64(data, version)` tasarım şablonuna uygun invoice yerleşimini çizer:

- Sol üst: kalın **INVOICE** başlığı (46 pt)
- Sağ üst: **URLA SHOES CORP.** + adres + email + telefon
- Meta satırı: Invoice #, Due Date, Invoice date, Prepared by
- Pembe banner: **CUSTOMER DETAILS** + Name / Address / Phone / Email
- Pembe başlıklı line item tablosu: *DESCRIPTION / QTY / PRICE / TOTAL*. Description boşsa `Product2.Name` kullanılır.
- Sol alt: Terms & Conditions paragrafı
- Sağ alt: SUBTOTAL / DISCOUNT / TAXES / GRAND TOTAL
- Sayfa altı: Signature / Name / Date satırları

Çıktı: PDF dokümanı → base64 string olarak Apex'e teslim edilir.

7 Apex Attachment kaydı oluşturur

```
@AuraEnabled
public static Id uploadPdf(Id quoteId, String base64Body, String fileName) {
    Attachment a = new Attachment(
        ParentId      = quoteId,
        Name           = fileName,                      // örn. "00000006_v2.pdf"
        Body           = EncodingUtil.base64Decode(base64Body),
        ContentType    = 'application/pdf'
    );
    insert a;
    return a.Id;
}
```

Yeni PDF **Notes & Attachments** related list'inde anında belirir.

8 Bir sonraki polling cycle baseline'ı günceller

Üretim tamamlandıktan sonraki `pollOnce`, artık güncel olan signature'ı görüp yeni baseline olarak saklar. Böylece **bir save tam olarak bir PDF üretir** — sonsuz döngü oluşmaz.

3. Versiyonlama Mantığı

```
@AuraEnabled
public static Integer getNextVersionNumber(Id quoteId) {
    Integer maxVersion = 0;
    for (Attachment a : [SELECT Name FROM Attachment
                          WHERE ParentId=:quoteId AND ContentType='application/pdf']) {
        Integer v = parseVersion(a.Name); // regex: _v(\d+)\.pdf
        if (v > maxVersion) maxVersion = v;
    }
    return maxVersion + 1;
}
```

Quote'a bağlı mevcut tüm PDF attachment'lar arasında en yüksek `_vN.pdf` suffix'ini bulup `N + 1` döner. İlk save'de v1, ikincide v2, sonrası benzeri.

4. Latency (Gecikme) Bütçesi

Aşama	Süre
Kullanıcı Save'e basar → DB committed	~0.5 sn
Bir sonraki polling cycle (en kötü ihtimal)	3 sn
Debounce penceresi	1.5 sn
PDF render + upload	~0.3 sn

En kötü senaryoda toplam ≈ 5 saniye; Save anından PDF'in Notes & Attachments'a düşmesine kadar

5. Temel Mimari Kararlar

5.1 — Neden Apex trigger değil de polling?

Visualforce kapsam dışı bırakılınca sunucu tarafı PDF render imkânsız hale geliyor. Üretim mecburen tarayıcıda (jsPDF ile) yapılmak zorunda; bu da otomasyonun LWC içinde yaşaması anlamına geliyor. İlk denemede `getRecord` (Lightning Data Service) kullanıldı, ancak QuoteLineItem rollup'larında tutarsız davrandı. Apex signature'ını 3 saniyede bir polling yapmak hem en güvenilir hem de tüm değişiklik tiplerini yakalayan yöntem oldu.

5.2 — Neden `getQuoteData` `cacheable=false` ?

İlk versiyonda `cacheable=true` idi. LWC save'den hemen sonra veri çektiği için cache eski (boş) veriyi dönüyordu ve PDF line item'ları görüntülemiyordu. Cache'i kaldırınca her üretimde taze veri çekilmesi sağlandı — geliştirme sırasında karşılaşılan ana hatanın kök nedeni buydu.

5.3 — Neden 3 sn polling, neden 1.5 sn debounce?

3 saniye kullanıcı için neredeyse anlık his veriyor; dakikada ~20 Apex çağrısı ise sistemde hissedilmeyecek kadar küçük yük. 1.5 sn'lik debounce ise wizard kaynaklı DML patlamalarını (Add Products'ın 5 line item eklemesi gibi) tek PDF'e indiriyor.

5.4 — Neden Files yerine Notes & Attachments?

Orijinal gereksinim açıkça "Notes & Attachments" diye belirtmişti. Klasik `Attachment` sObject'i tam olarak buraya düşüyor ve şartı birebir karşılıyor. `ContentDocument` (Files) daha modern bir alternatif olur; ileride takım isterse tek satırlık bir değişiklik ile geçilebilir.

6. Hata Senaryoları

Senaryo	Sonuç
jsPDF static resource yüklenemezse	Card'da kırmızı hata mesajı: <i>"jsPDF library failed to load"</i> . Polling hiç başlamaz.
<code>getQuoteData</code> hata fırlatırsa	Toast: <i>"PDF generation failed"</i> + hata mesajı; kullanıcı "Regenerate Now" butonuyla tekrar deneyebilir.
<code>uploadPdf</code> FLS / sharing nedeniyle engellenirse	Toast Apex hatasını gösterir; koşullar değişirse bir sonraki polling cycle'da yeniden denir.

Aynı anda iki üretim tetiklenirse	<code>isGenerating</code> guard'ı erken return ettirir; yalnızca ilk üretim tamamlanır.
Polling sırasında ağ kopukluğu	Sessiz hata — 3 saniye sonraki poll tekrar dener.
Kullanıcı debounce sırasında 3 save daha yaparsa	Her save timer'ı sıfırlar; yalnızca son save'den 1.5 sn sonra tek bir PDF üretilir.

7. Dosya Yapısı

```

force-app/main/default/
├── classes/
│   ├── QuotePdfService.cls           X- 4 Apex metodu
│   └── QuotePdfServiceTest.cls       X- 6 unit test, %100 coverage
├── lwc/
│   └── quotePdfGenerator/
│       ├── quotePdfGenerator.js      X- ana mantık (polling, render, upload)
│       ├── quotePdfGenerator.html    X- UI template
│       ├── quotePdfGenerator.css     X- stil
│       └── quotePdfGenerator.js-meta.xml X- Quote record page target
├── staticresources/
│   ├── jsPDF.js                     X- jsPDF 2.5.1 UMD min (365 KB)
│   └── jsPDF.resource-meta.xml
└── settings/
    └── Quote.settings-meta.xml       X- Quote objesini aktive eder

```

8. Takım İçin 3 Anahtar Mesaj

1. **"Visualforce kullanmadan otomatik PDF üretimi tarayıcıya taşınır."** — modern, client-side, sunucu tarafı render bağımlılığı yok.
2. **"Tetikleyici bir Apex trigger değil; 3 saniyelik polling + 1.5 saniyelik debounce."** — basit, öngörülebilir ve farklı değişiklik tiplerini güvenilir biçimde yakalar.
3. **"`cacheable=false` + her seferinde taze veri çekme = doğru PDF."** — geliştirme sırasındaki boş PDF hatasının kök nedeni cacheable Apex idi; çözüm bilinçli ve takımla paylaşılmaya değer.

Takım inceleme için hazırlanmıştır — URLA SHOES Salesforce Sandbox